F1 Insight API — Documentation (v5.0.0)

1) Overview

F1 Insight API is a FastAPI + SQLite REST API built on an Ergast-style historical Formula 1 dataset. It exposes:

- **Read-only** endpoints for F1 reference data (drivers, constructors, races, results, standings).
- A full **CRUD** resource (`notes`) that is writable and stored in SQLite.
- A deterministic analytics layer ("**fact packs**") that structures race/season data for insight generation.
- An **insights** layer that can generate narrative output via:
    - `generator=template` (deterministic)
    - `generator=llm` (local LLM via `generate_llm_insight`)

The API uses FastAPI validation for query parameters and returns standard JSON responses.

---

2) Base URL & Interactive Docs

Base URL:
`http://127.0.0.1:8000`

Swagger UI:
`http://127.0.0.1:8000/docs`

---

3) Startup / Lifespan Behaviour

On startup, the API initialises the writable model:

- `ensure_notes_table(db)`
- `ensure_notes_ai_columns(db)`

This ensures the `notes` table exists and has AI-related columns (e.g., `ai_summary`) where needed.

---

4) Authentication

None required in the current implementation.
(There is no authentication middleware or API key checks in the provided code.)

---

5) Common Conventions

Pagination

Many list endpoints support:

- `limit`: integer, default varies, `1 ≤ limit ≤ 200`
- `offset`: integer, default `0`, `offset ≥ 0`

Responses often include:

- `count`: number of items returned in this page (not total count)
- `limit`, `offset`
- `results`: list payload

Insight parameters

Insights endpoints accept:

- `mode`: `recap` or `impact`
- `format`: `plain` or `radio`
  (implemented internally as `fmt`, with FastAPI alias `format`)
- `generator`: `template` or `llm`

---

6) Error Codes

Common HTTP status codes:

- **200 OK** – successful read/processing
- **201 Created** – resource created (POST notes / save insight as note)
- **204 No Content** – successful delete (DELETE notes)
- **400 Bad Request** – invalid input / validation error / year out of range
- **404 Not Found** – requested resource not found

FastAPI validation failures may return **422 Unprocessable Entity** automatically (e.g., wrong type, invalid pattern), depending on what fails validation.

---

7) Endpoints

7.1 Health

GET `/health`

Description: Health check and DB path info.

Response (200)

{

```
 "status": "ok",
 "db_file": "path/to/db.sqlite"
}
```

7.2 Drivers

GET `/drivers`

Description: List drivers.

Query parameters

- `limit` (int, default 50, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)


```
{
 "count": 2,
 "limit": 50,
 "offset": 0,
 "results": [
   {
     "driverId": 1,
     "forename": "Lewis",
     "surname": "Hamilton",
     "nationality": "British",
     "dob": "1985-01-07"
   }
 ]
}
```

GET `/drivers/{driverId}`

Description: Get one driver by ID.

Path parameters

- `driverId` (int)

Response (200)


```
{
 "driverId": 1,
 "driverRef": "hamilton",
 "number": 44,
 "code": "HAM",
 "forename": "Lewis",
 "surname": "Hamilton",
 "dob": "1985-01-07",
 "nationality": "British",
 "url": "https://..."
}
```


Errors

- `404` with:


```
{ "detail": "Driver not found" }
```

7.3 Constructors

GET `/constructors`

Description: List constructors.

Query parameters

- `limit` (int, default 50, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)


```
{
 "count": 2,
 "limit": 50,
 "offset": 0,
 "results": [
   {
     "constructorId": 1,
```

```
    "name": "Mercedes",
    "nationality": "German"
  }
 ]
}
```

GET `/constructors/{constructorId}`

Description: Get one constructor by ID.

Path parameters

- `constructorId` (int)

Response (200)


```
{
 "constructorId": 1,
 "constructorRef": "mercedes",
 "name": "Mercedes",
 "nationality": "German",
 "url": "https://..."
}
```


Errors

- `404`


```
{ "detail": "Constructor not found" }
```

7.4 Races

GET `/races`

Description: List races for a given season year.
If `year` is not provided, the API defaults to the most recent year in the DB.

Query parameters

- `year` (int, optional, `>= 1950`)
- `limit` (int, default 50, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)


```
{
 "year": 2024,
 "count": 2,
 "limit": 50,
 "offset": 0,
 "results": [
  {
   "raceId": 1100,
   "year": 2024,
   "round": 1,
   "name": "Bahrain Grand Prix",
   "date": "2024-03-02",
   "time": "15:00:00",
   "circuitId": 3
  }
 ]
}
```

GET `/races/{raceId}`

Description: Race metadata + circuit details.

Path parameters

- `raceId` (int)

Response (200)


```
{
 "raceId": 1100,
 "year": 2024,
 "round": 1,
 "raceName": "Bahrain Grand Prix",
```

```
 "date": "2024-03-02",
 "time": "15:00:00",
 "url": "https://...",
 "circuitId": 3,
 "circuitName": "Bahrain International Circuit",
 "location": "Sakhir",
 "country": "Bahrain"
}
```


Errors

- `404`


```
{ "detail": "Race not found" }
```

GET `/races/{raceId}/results`

Description: Results for a race (paginated).

Path parameters

- `raceId` (int)

Query parameters

- `limit` (int, default 50, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)


```
{
 "race": {
   "raceId": 1100,
   "year": 2024,
   "round": 1,
   "name": "Bahrain Grand Prix",
   "date": "2024-03-02"
 },
 "count": 2,
 "limit": 50,
```

```
 "offset": 0,
 "results": [
   {
     "resultId": 1,
     "positionOrder": 1,
     "points": 25,
     "grid": 1,
     "laps": 57,
     "time": "1:31:44.000",
     "milliseconds": 5500000,
     "driverId": 830,
     "driverName": "Max Verstappen",
     "constructorId": 9,
     "constructorName": "Red Bull",
     "status": "Finished"
   }
 ]
}
```


Errors

- `404`

{ "detail": "Race not found" }

7.5 Standings

GET `/seasons/{year}/driver-standings`

Description: Driver standings for a season (computed using SUM(points)).

Path parameters

- `year` (int)

Query parameters

- `limit` (int, default 30, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)


```
{
 "year": 2024,
 "count": 2,
 "limit": 30,
 "offset": 0,
 "results": [
   {
     "driverId": 830,
     "driverName": "Max Verstappen",
     "nationality": "Dutch",
     "points": 430.0,
     "wins": 14,
     "starts": 24,
     "rank": 1
   }
 ]
}
```


Errors

- `400` if year out of range:


```
{ "detail": "year must be between 1950 and 2024" }
```

GET `/seasons/{year}/constructor-standings`

Description: Constructor standings for a season (computed using SUM(points)).

Path parameters

- `year` (int)

Query parameters

- `limit` (int, default 20, min 1, max 200)
- `offset` (int, default 0, min 0)

Response (200)

```
{
 "year": 2024,
 "count": 1,
 "limit": 20,
 "offset": 0,
 "results": [
  {
   "constructorId": 9,
   "constructorName": "Red Bull",
   "nationality": "Austrian",
   "points": 860.0,
   "wins": 18,
   "starts": 48,
   "rank": 1
  }
 ]
}
```


Errors

- `400` year out of range:


```
{ "detail": "year must be between 1950 and 2024" }
```

7.6 Driver Season Summary

GET `/drivers/{driverId}/seasons/{year}`

Description: Aggregated stats for a driver in a given season.
Optional: include per-race results.

Path parameters

- `driverId` (int)
- `year` (int)

Query parameters

- `include_results` (bool, default `false`)

Response (200)


```
{
 "driver": { "driverId": 830, "driverName": "Max Verstappen" },
 "year": 2024,
 "points": 430.0,
 "wins": 14,
 "podiums": 18,
 "starts": 24,
 "results": [
  {
   "raceId": 1100,
   "round": 1,
   "raceName": "Bahrain Grand Prix",
   "positionOrder": 1,
   "points": 25,
   "constructorName": "Red Bull",
   "status": "Finished"
  }
 ]
}
```


If `include_results=false`, the `results` field is omitted.

Errors

- `404`

{ "detail": "Driver not found" }

- `400` year out of range:

{ "detail": "year must be between 1950 and 2024" }

7.7 Debug Fact Packs

These endpoints expose deterministic analytics outputs used to ground insights.

GET `/debug/races/{raceId}/facts`

Description: Build race fact pack.

Errors

- `404`

{ "detail": "Race not found" }

GET `/debug/seasons/{year}/facts`

Description: Build season fact pack.

Errors

- `400` year out of range:

{ "detail": "year must be between 1950 and 2024" }

- `404`

{ "detail": "Season not found" }

GET `/debug/seasons/{year}/compare/{compare_to}`

Description: Build a season comparison fact pack.

Errors

- `400` year out of range:

{ "detail": "year must be between 1950 and 2024" }

- `404`

{ "detail": "Comparison not available" }

7.8 Insights

Insights return both the underlying `facts` and the generated `insight`.

GET `/races/{raceId}/insights`

Description: Generate a race insight from deterministic race facts.

Path parameters

- `raceId` (int)

Query parameters

- `mode` (string, default `"recap"`, must match `^(recap|impact)$`)
- `format` (string, default `"plain"`, must match `^(plain|radio)$`)
- `generator` (string, default `"template"`, must match `^(template|llm)$`)

> Implementation detail: `format` is accepted as a query parameter but stored internally as `fmt` via `alias="format"`.

Response (200)


```
{
 "raceId": 1100,
 "mode": "recap",
 "format": "radio",
 "generator": "template",
 "facts": { "...": "..." },
 "insight": "..."
}
```


Errors

- `404`

{ "detail": "Race not found" }

GET `/seasons/{year}/insights`

Description: Generate a season insight. If `compare_to` is provided, returns a comparison insight.

Path parameters

- `year` (int)

Query parameters

- `compare_to` (int, optional)
- `mode` (string, default `"recap"`, pattern `^(recap|impact)$`)
- `format` (string, default `"plain"`, pattern `^(plain|radio)$`)
- `generator` (string, default `"template"`, pattern `^(template|llm)$`)

Response (200), single season

```
{
 "year": 2024,
 "mode": "impact",
 "format": "plain",
 "generator": "llm",
 "facts": { "...": "..." },
 "insight": "..."
}
```

Response (200), comparison

```
{
 "year": 2024,
 "compare_to": 2023,
 "mode": "recap",
 "format": "radio",
 "generator": "template",
 "facts": { "...": "..." },
 "insight": "..."
}
```

Errors

- `400` year out of range:

{ "detail": "year must be between 1950 and 2024" }

- `404` season not found:

{ "detail": "Season not found" }

- `404` comparison not available:

{ "detail": "Comparison not available" }

7.9 Notes (CRUD)

Notes are a writable model designed for user content and persistent AI outputs.

Data Model

Create request (`NoteCreate`)

- `entity_type`: `"race" | "driver" | "season"` (required)
- `entity_id`: int, `>= 1` (required)
- `title`: string, length `1..120` (required)
- `content`: string, length `1..5000` (required)

Update request (`NoteUpdate`)

- `title`: optional, length `1..120`
- `content`: optional, length `1..5000`
- At least one must be provided

POST `/notes` (201)

Description: Create a note.

Request body


```
{
 "entity_type": "race",
 "entity_id": 1100,
 "title": "My note",
 "content": "Great race..."
}
```


Response (201)


```
{
 "id": 1,
 "entity_type": "race",
 "entity_id": 1100,
 "title": "My note",
 "content": "Great race...",
 "created_at": "2026-03-05 12:00:00",
 "updated_at": "2026-03-05 12:00:00"
}
```

GET `/notes`

Description: List notes, optionally filtered.

Query parameters

- `entity_type` (optional, must match `^(race|driver|season)$`)
- `entity_id` (optional int, `>=1`)
- `limit` (int, default 50, 1..200)
- `offset` (int, default 0, >=0)

Response (200)


```
{
 "count": 1,
```

```
 "limit": 50,
 "offset": 0,
 "results": [
  {
    "id": 1,
    "entity_type": "race",
    "entity_id": 1100,
    "title": "My note",
    "content": "Great race...",
    "created_at": "2026-03-05 12:00:00",
    "updated_at": "2026-03-05 12:00:00"
  }
 ]
}
```

GET `/notes/{note_id}`

Description: Get a note by ID.

Errors

- `404`


```
{ "detail": "Note not found" }
```

PUT `/notes/{note_id}`

Description: Update a note.

Request body


```
{
 "title": "Updated title",
 "content": "Updated content"
}
```


Errors

- `404`


```
{ "detail": "Note not found" }
```

- `400`

{ "detail": "No fields provided to update" }

DELETE `/notes/{note_id}` (204)

Description: Delete a note.

Response

- `204 No Content`

Errors

- `404`

{ "detail": "Note not found" }

7.10 AI + Notes Workflows

POST `/notes/{note_id}/ai/tldr`

Description: Generate a TL;DR summary for a note and store it in `ai_summary`.

Query parameters

- `generator` (string, default `"llm"`, pattern `^(template|llm)$`)

Behaviour

- If `generator=template`: deterministic summary = first 240 characters (+ `...` if truncated)
- If `generator=llm`: uses `generate_llm_insight` with facts:

  { "type": "user_note", "title": "...", "content": "..." }

and calls it with `mode="recap"` and `fmt="plain"`.

Errors

- `404` note not found

{ "detail": "Note not found" }

- `400` note content empty

{ "detail": "Note content is empty" }

Response (200)


```
{
 "generator": "llm",
 "note": {
   "id": 1,
   "entity_type": "race",
   "entity_id": 1100,
   "title": "My note",
   "content": "Great race...",
   "ai_summary": "Short summary...",
   "created_at": "2026-03-05 12:00:00",
   "updated_at": "2026-03-05 12:05:00"
 }
}
```

POST `/races/{raceId}/notes` (201)

Description: Generate a race insight and save it as a note linked to that race.

Request body (`SaveRaceInsightRequest`)

- `title` (optional)
- `mode` = `"recap" | "impact"` (default `recap`)
- `format` = `"plain" | "radio"` (default `plain`)

- `generator` = `"template" | "llm"` (default `template`)

Response (201)


```
{
 "raceId": 1100,
 "saved_note": {
   "id": 12,
   "entity_type": "race",
   "entity_id": 1100,
   "title": "2024 Bahrain Grand Prix – recap (radio)",
   "content": "...",
   "created_at": "2026-03-05 12:10:00",
   "updated_at": "2026-03-05 12:10:00"
 },
 "generator": "llm",
 "mode": "recap",
 "format": "radio"
}
```


Errors

- `404`

{ "detail": "Race not found" }

POST `/seasons/{year}/notes` (201)

Description: Generate a season insight (optionally a comparison) and save it as a note linked to the season year.

Request body (`SaveSeasonInsightRequest`)

- `title` (optional)
- `compare_to` (optional int)
- `mode` = `"recap" | "impact"` (default `recap`)
- `format` = `"plain" | "radio"` (default `plain`)

- `generator` = `"template" | "llm"` (default `template`)

Response (201), single season


```
{
 "year": 2024,
 "saved_note": {
   "id": 13,
   "entity_type": "season",
   "entity_id": 2024,
   "title": "2024 – impact (plain)",
   "content": "...",
   "ai_summary": null,
   "created_at": "2026-03-05 12:20:00",
   "updated_at": "2026-03-05 12:20:00"
 },
 "generator": "template",
 "mode": "impact",
 "format": "plain"
}
```


Response (201), comparison


```
{
 "year": 2024,
 "compare_to": 2023,
 "saved_note": {
   "id": 14,
   "entity_type": "season",
   "entity_id": 2024,
   "title": "2024 vs 2023 – recap (radio)",
   "content": "...",
   "ai_summary": null,
   "created_at": "2026-03-05 12:25:00",
   "updated_at": "2026-03-05 12:25:00"
 },
 "generator": "llm",
 "mode": "recap",
 "format": "radio"
}
```


Errors

- `400` year out of range

{ "detail": "year must be between 1950 and 2024" }

- `404` comparison not available

{ "detail": "Comparison not available" }

- `404` season not found (single season mode)

{ "detail": "Season not found" }

---

8) Example Request Set (Quick Testing)

Health

curl http://127.0.0.1:8000/health

Drivers

curl "http://127.0.0.1:8000/drivers?limit=5&offset=0"
curl "http://127.0.0.1:8000/drivers/1"

Races (latest year)

curl "http://127.0.0.1:8000/races?limit=5"

Race results

curl "http://127.0.0.1:8000/races/1100/results?limit=10"

Insights (template)

curl "http://127.0.0.1:8000/races/1100/insights?mode=recap&format=radio&generator=template"
curl
"http://127.0.0.1:8000/seasons/2024/insights?mode=impact&format=plain&generator=template"

Insights (LLM)

```
curl "http://127.0.0.1:8000/races/1100/insights?mode=recap&format=radio&generator=llm"
```

Notes CRUD

```
curl -X POST "http://127.0.0.1:8000/notes" \
 -H "Content-Type: application/json" \
 -d '{"entity_type":"race","entity_id":1100,"title":"My note","content":"Great race"}'

curl "http://127.0.0.1:8000/notes?limit=10"
curl "http://127.0.0.1:8000/notes/1"

curl -X PUT "http://127.0.0.1:8000/notes/1" \
 -H "Content-Type: application/json" \
 -d '{"title":"Updated title"}'

curl -X DELETE "http://127.0.0.1:8000/notes/1"
```

AI TL;DR

```
curl -X POST "http://127.0.0.1:8000/notes/1/ai/tldr?generator=template"
curl -X POST "http://127.0.0.1:8000/notes/1/ai/tldr?generator=llm"
```

Save insights as notes

```
curl -X POST "http://127.0.0.1:8000/races/1100/notes" \
 -H "Content-Type: application/json" \
 -d '{"mode":"recap","format":"radio","generator":"template"}'

curl -X POST "http://127.0.0.1:8000/seasons/2024/notes" \
 -H "Content-Type: application/json" \
 -d '{"compare_to":2023,"mode":"impact","format":"plain","generator":"llm"}'
```

9) Frontend User Interface

The F1 Insight API also includes a lightweight **React-based frontend client (apiV8)** that allows users to interact with the API through a graphical interface.

The frontend is implemented using:

- **React + Vite**
- **React Router**

- **Fetch API for HTTP requests**
- Modular UI components (Card, Button, Input)

The interface is designed to demonstrate how the API can be consumed by a client application and provides interactive access to the major API endpoints.

Key Features

The interface contains several sections corresponding to API functionality:

Page	Description
Drivers	Search and browse all drivers in the dataset
Races	View races by season
Results	View race results for a selected race
Standings	Retrieve championship standings for a given season
Insights	Generate race or season insights using template or LLM generation
Notes	Create, edit, and delete user notes stored in the API

Driver Search

The Drivers page implements **client-side filtering** that allows users to search the full driver dataset by:

- driver name
- surname
- nationality

This provides a more usable interface for exploring the dataset.

API Health Monitoring

The UI also includes an **API Status panel** which calls the `/health` endpoint to verify that the backend service is running and connected to the database.

Example response:

{
 "status": "ok",
 "db_file": "f1.sqlite3"
}

Purpose of the Frontend

The frontend serves three main purposes:

1. Demonstrates **real-world API consumption**
2. Provides an **interactive exploration tool** for the dataset
3. Acts as a **visual testing interface** for API endpoints

The UI is intentionally lightweight and focuses on interacting with the API rather than replacing the API documentation.